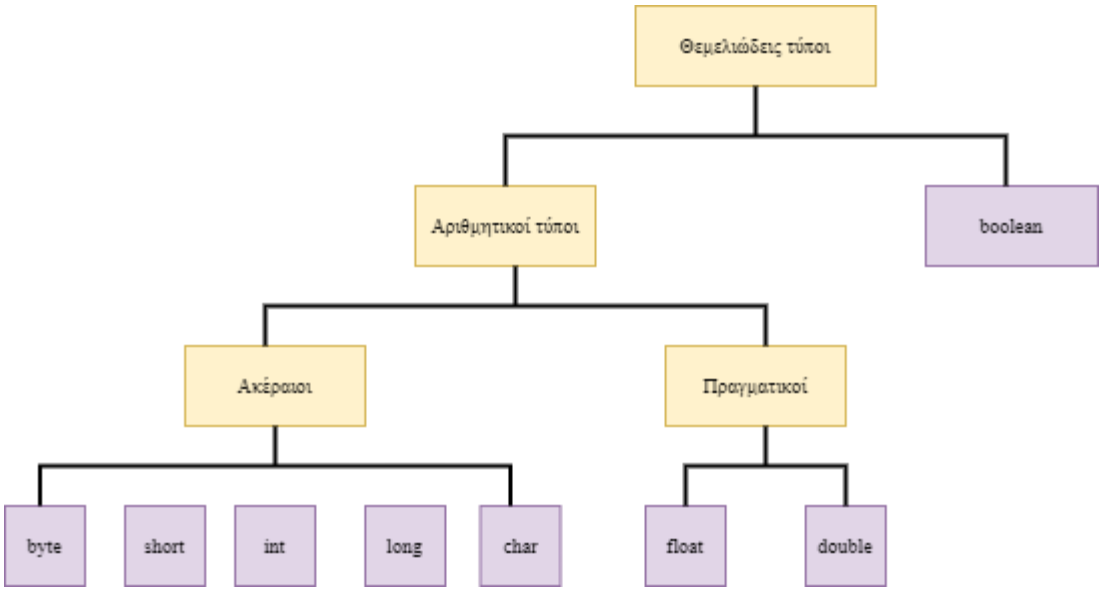
Όπως φαίνεται από τα παραδείγματα αυτής της ενότητας, οι συμβάσεις ονοματολογίας για τις σταθερές διαφέρουν από τις αντίστοιχες συμβάσεις για μεταβλητές. Οι σταθερές ονομάζονται αποκλειστικά με χρήση κεφαλαίων χαρακτήρων.

3.2 Θεμελιώδεις τύποι δεδομένων

Όπως εξηγήσαμε στην προηγούμενη ενότητα, τα προγράμματα διαχειρίζονται τα δεδομένα τους με τη βοήθεια των μεταβλητών. Τα δεδομένα είναι διάφορων τύπων, π.χ. ακέραιοι αριθμοί, πραγματικοί, χαρακτήρες, κλπ. Αντίστοιχα, και οι μεταβλητές που χρησιμοποιούνται τυποποιούνται ως ακέραιες, πραγματικές, μεταβλητές χαρακτήρων, κλπ. Η Java θέτει αυστηρούς κανόνες σχετικά με τον προσδιορισμό τύπου μιας μεταβλητής. Πιο συγκεκριμένα, απαιτεί ο προσδιορισμός τύπου κάθε μεταβλητής να είναι δεδομένος κατά τον χρόνο μεταγλώττισης και να παραμένει αμετάβλητος για όλη τη ζωή της μεταβλητής. Γλώσσες με παρόμοια συμπεριφορά χαρακτηρίζονται ως γλώσσες ισχυρού τύπου (Strongly typed languages), σε αντίθεση με τις γλώσσες που δεν θέτουν αυτές τις απαιτήσεις και χαρακτηρίζονται ως γλώσσες αδύναμου τύπου (Weakly typed languages).

Ένα επίσης σημαντικό χαρακτηριστικό της Java είναι η πρώιμη διασύνδεση (early binding). Όπως αναφέραμε μια μεταβλητή μπορεί να φιλοξενεί μια ακέραιη τιμή ή έναν χαρακτήρα. Και στις δύο περιπτώσεις όμως τα δεδομένα στη μνήμη αποθηκεύονται ως μια σειρά από δυαδικά ψηφία. Για να είναι δυνατή η ορθή ερμηνεία αυτών των δεδομένων είναι αναγκαίο να γνωρίζουμε τον τύπο της μεταβλητής. Με άλλα λόγια, όταν προσπελαύνουμε μια θέση μνήμης, δηλαδή μια σειρά από bits, πρέπει να είναι γνωστό αν η σειρά αυτή αναπαριστά έναν ακέραιο ή έναν χαρακτήρα. Η Java χαρακτηρίζει κάθε μεταβλητή με συγκεκριμένο τύπο, γνωστό κατά τον χρόνο μεταγλώττισης και σταθερό καθόλη την εκτέλεση του προγράμματος. Έτσι όταν προσπελαύνει μια μεταβλητή γνωρίζοντας τη διεύθυνσή της στη μνήμη και τον τύπο της μπορεί άμεσα να την αξιοποιήσει. Στον αντίποδα, οι γλώσσες καθυστερημένης διασύνδεσης (late binding), επιτρέπουν κατά τη διάρκεια εκτέλεσης του προγράμματος την εκχώρηση στην ίδια μεταβλητή δεδομένων ποικίλων τύπων. Για παράδειγμα, η Javascript, επιτρέπει την εκχώρηση σε μεταβλητή μιας ακέραιας τιμής και παρακάτω στον ίδιο κώδικα την εκχώρηση μιας συμβολοσειράς. Επομένως, ο τύπος της μεταβλητής δεν μπορεί να καθοριστεί παρά μόνο κατά τον χρόνο εκτέλεσης. Έτσι όταν η Javascript προσπελαύνει μια μεταβλητή θα πρέπει να υπολογίσει ποιος είναι ο τύπος της σε ακριβώς εκείνο το σημείο της εκτέλεσης. Όπως είναι φανερό, η πρώιμη διασύνδεση βελτιώνει την ταχύτητα εκτέλεσης.

Οι μεταβλητές επομένως στην Java έχουν προκαθορισμένο τύπο. Οι διαθέσιμοι τύποι είναι πάρα πολλοί. Επιπλέον, ο χρήστης έχει τη δυνατότητα να ορίσει και δικούς του τύπους (User Defined Types). Σε αυτήν την ενότητα όμως παρουσιάζουμε ειδικά τους θεμελιώδεις τύπους (primitive types) της Java. Οι θεμελιώδεις τύποι είναι οκτώ [1], είναι προκαθορισμένοι (predefined) και τα ονόματά τους αποτελούν δεσμευμένες λέξεις (reserved words). Πλήρη λίστα με τις λέξεις κλειδιά της Java μπορείτε να βρείτε στο παράρτημα 2. Όλοι οι υπόλοιποι τύποι προκύπτουν από συνδυασμό ή επανορισμό των θεμελιωδών τύπων. Στο σχήμα 3.1, παρουσιάζεται η κατηγοριοποίηση των θεμελιωδών τύπων.



***Σχήμα 3.1**: Κατηγοριοποίηση των θεμελιωδών τύπων*

byte Ο τύπος byte είναι προσημασμένος ακέραιος μήκους οκτώ δυαδικών ψηφίων (bits). Αν η τιμή μιας μεταβλητής τύπου byte είναι θετική, τότε αναπαρίσταται στη μνήμη κωδικοποιημένη με τη μορφή πρόσημο και μέτρο (sign and magnitude), δηλαδή το πρώτο bit έχει την τιμή 0 για να δείξει πως ο αριθμός είναι θετικός και τα υπόλοιπα bits διατηρούν την τιμή του αριθμού εκφρασμένη στο δυαδικό σύστημα. Αν η τιμή μιας μεταβλητής τύπου byte είναι αρνητική, αυτή αναπαρίσταται στη μνήμη με τη μορφή συμπληρώματος ως προς 2 (two's complement) (παράρτημα 1). Σε αυτήν την περίπτωση το πρώτο bit έχει την τιμή 1 που δείχνει πως ο αριθμός είναι αρνητικός. Επομένως, τόσο για τους θετικούς όσο και για τους αρνητικούς, εξαιρώντας το πρώτο bit που ονομάζεται και bit πρόσημου, διαπιστώνουμε πως για την αναπαράσταση της τιμής απομένουν 7 bits. Συνεπώς, αναπαρίστανται συνολικά 2^7 αρνητικές τιμές και 2^7 μη αρνητικές τιμές. Πιο συγκεκριμένα, αναπαρίστανται οι αρνητικές τιμές $-1..-2^7$ και οι μη αρνητικές τιμές $0..2^7-1$. Προσέξτε πως η μικρότερη αρνητική τιμή, δηλαδή το -2^7, είναι κατ' απόλυτη τιμή μεγαλύτερη κατά μια μονάδα από τη μεγαλύτερη μη αρνητική, δηλαδή το 2^7-1. Αυτό συμβαίνει γιατί στις μη αρνητικές τιμές συμπεριλαμβάνεται και το 0.

Η Java για κάθε θεμελιώδη τύπο διαθέτει και μια κλάση (Class) που παρέχει λειτουργίες σχετικές με τον θεμελιώδη τύπο. Την έννοια της κλάσης θα αναλύσουμε με λεπτομέρεια κατά τη μελέτη του αντικειμενοστρεφούς μοντέλου. Για την ώρα, θα θεωρούμε την κλάση ως ένα σύνθετο τύπο ο οποίος, εκτός από δεδομένα, ενσωματώνει και λειτουργίες (συναρτήσεις). Σε γενικές γραμμές, θεμελιώδεις τύποι και αντίστοιχες κλάσεις έχουν την ίδια ονομασία με μόνη διαφορά ότι το όνομα της κλάσης ξεκινάει με κεφαλαίο ενώ του τύπου με πεζό. Έτσι ο τύπος byte συνοδεύεται από την κλάση Byte.

```
static void byteDemo() {
        System.out.println("Size of byte is " + Byte.SIZE + " bits");
        byte bMin = Byte.MIN_VALUE, bMax = Byte.MAX_VALUE, bLiteral = 123;
        System.out.println("bMin equals to " + bMin);
        System.out.println("bMax equals to " + bMax);
        System.out.println("bLiteral equals to " + bLiteral);
}
```

***Κώδικας 3.1**: Παράδειγμα χρήσης μεταβλητής τύπου byte*

Στον κώδικα 3.1, καταρχάς, εμφανίζουμε το μέγεθος του τύπου (Byte.SIZE) μετρημένο σε δυαδικά ψηφία (bits). Μετά δηλώνουμε τις μεταβλητές bMin, bMax και bLiteral. Την πρώτη αρχικοποιούμε στην ελάχιστη τιμή του τύπου (Byte.MIN_VALUE), τη δεύτερη στη μέγιστη τιμή του τύπου (Byte.MAX_VALUE) και

την τρίτη σε μια ενδιάμεση τιμή. Την ελάχιστη και μέγιστη τιμή μας παρέχει η κλάση Byte που τις περιλαμβάνει ως σταθερές μεταβλητές (variable constants). Για την ενδιάμεση τιμή χρησιμοποιούμε μια κυριολεκτική σταθερά (literal). Στη συνέχεια στέλνουμε στη συσκευή εξόδου, δηλαδή στην τυπική περίπτωση στην οθόνη, το περιεχόμενο των bMin, bMax και bLiteral.
Παρατίθεται η έξοδος του κώδικα 3.1.

```
Size of byte is 8 bits
bMin equals to -128
bMax equals to 127
bLiteral equals to 123
```

short Ο τύπος short είναι προσημασμένος ακέραιος. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Το μήκος του όμως είναι 16 δυαδικών ψηφίων. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου short είναι $0..2^{15}-1$ και $-1..-2^{15}$.
Στον κώδικα 3.2, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου short.

```
static void shortDemo() { //Κώδικας 3.2
        short s = Short.MIN_VALUE, s1 = Short.MAX_VALUE, s2 = 399;
        System.out.println("short s equals to " + s);
        System.out.println("short s1 equals to " + s1);
        System.out.println("short s2 equals to " + s2);
        System.out.println("Size of short is " + Short.SIZE + " bits");
}
```

***Κώδικας 3.2**: Παράδειγμα χρήσης μεταβλητών τύπου short*

Ακολουθεί η έξοδος του κώδικα 3.2

```
short s equals to -32768
short s1 equals to 32767
short s2 equals to 399
Size of short is 16 bits
```

int Ο τύπος int είναι προσημασμένος ακέραιος μήκους 32 δυαδικών ψηφίων. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου int είναι $0..2^{31}-1$ και $-1..-2^{31}$.
Στον κώδικα 3.3, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου int.

```
    static void intDemo() { //Κώδικας 3.3
        int iMin = Integer.MIN_VALUE,
            iMax = Integer.MAX_VALUE, iLiteral = 43127;
        System.out.println("int iMin equals to " + iMin);
        System.out.println("int iMax equals to " + iMax);
        System.out.println("int iLiteral equals to " + iLiteral);
        System.out.println("Size of int is " + Integer.SIZE + " bits");
    }
```

***Κώδικας 3.3**: Δήλωση και αρχικοποίηση μεταβλητών τύπου int*

Ακολουθεί η έξοδος του κώδικα 3.3.

```
int iMin equals to -2147483648
```

```
int iMax equals to 2147483647
int iLiteral equals to 43127
Size of int is 32 bits
```

Προσέξτε πως κατ' εξαίρεση, το όνομα της κλάσης που μας παρέχει πληροφορίες σχετικά με τον τύπο int, δεν είναι Int αλλά Integer.

long Ο τύπος long είναι προσημασμένος ακέραιος μήκους 64 δυαδικών ψηφίων. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου long είναι $0..^{263}-1$ και $-1..-2^{63}$.
Στον κώδικα 3.4, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου long.

```
static void longDemo() { //Κώδικας 3.4
    long lMin = Long.MIN_VALUE,
            lMax = Long.MAX_VALUE, lLiteral = 2_147_483_699L;
    System.out.println("long lMin equals to " + lMin);
    System.out.println("long lMax equals to " + lMax);
    System.out.println("long lLiteral equals to " + lLiteral);
    System.out.println("Size of long is " + Long.SIZE + " bits");
}
```

***Κώδικας 3.4**: Δήλωση και αρχικοποίηση μεταβλητών τύπου long*

Ακολουθεί η έξοδος του κώδικα 3.4.

```
long lMin equals to -9223372036854775808
long lMax equals to 9223372036854775807
long lLiteral equals to 2147483699
Size of long is 64
```

Προσέξτε πως στην αρχικοποίηση της lLiteral, η κυριολεκτική σταθερά ακολουθείται από τον χαρακτήρα L. Οι ακέραιες κυριολεκτικές σταθερές που δεν ακολουθούνται από το L έχουν τύπο int. Αν θέλουμε να χρησιμοποιήσουμε κυριολεκτικές σταθερές με τύπο long, θα πρέπει αυτές να ακολουθούνται από το L. Στον κώδικα 3.4, αν αφαιρέσουμε το L, ο μεταγλωττιστής θα παράξει error, καθώς η τιμή 2147483699 είναι μεγαλύτερη από τη μέγιστη τιμή τύπου int. Ωστόσο σταθερές τύπου int (μέσα στα όρια του τύπου int) μπορούν να εκχωρηθούν σε μεταβλητές τύπου long, καθώς η Java κάνει αυτόματες μετατροπές μεταξύ συμβατών τύπων. Εναλλακτικά, αντί για κεφαλαίο L μπορεί να χρησιμοποιηθεί το πεζό l. Ωστόσο, συνίσταται η χρήση του κεφαλαίου καθώς το πεζό διακρίνεται με δυσκολία από τον αριθμό 1.

Προσέξτε επίσης πως ανάμεσα στα ψηφία έχουμε τοποθετήσει χαρακτήρες '_' (underscore). Από την έκδοση 7 και μετά, η Java επιτρέπει οποιονδήποτε αριθμό από underscores σε αριθμητικές σταθερές ώστε να διευκολύνεται η αναγνωσιμότητα των προγραμμάτων. Ωστόσο, δεν επιτρέπεται ο χαρακτήρας underscore, στην αρχή ή στο τέλος ενός αριθμού.

float Ο τύπος float χρησιμοποιείται για διαχείριση πραγματικών αριθμών. Έχει μήκος 32 δυαδικά ψηφία. Η κωδικοποίηση στη μνήμη των τιμών αυτού του τύπου ακολουθεί ένα πρότυπο γνωστό ως IEEE 754 και αναπτύχθηκε από το Ινστιτούτο Ηλεκτρολόγων και Ηλεκτρονικών Μηχανικών (Institute of Electrical and Electronics Engineers), που εδρεύει στη Νέα Υόρκη και στο οποίο συνήθως αναφερόμαστε με την αγγλική συντομογραφία IEEE. Η ανάλυση του προτύπου IEEE 754 είναι έξω από τα όρια αυτού του εγχειριδίου. Ωστόσο, είναι χρήσιμο να γνωρίζει κανείς πως οι πραγματικοί αριθμοί αναπαρίστανται σε εκθετική μορφή (Exponential Notation) και εκφράζονται με επιστημονικό συμβολισμό (scientific notation), mxE^n, όπου m ένας πραγματικός αριθμός, n ακέραιος και E=10. Οι float αριθμοί χαρακτηρίζονται ως αριθμοί κινητής υποδιαστολής απλής ακρίβειας (single-precision floating point numbers).

Στον κώδικα 3.5, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου float.

```
static void floatDemo() { //Κώδικας 3.5
        float fMin = Float.MIN_VALUE, fMax = Float.MAX_VALUE, fLit = 3.15f;
        System.out.println("float fMin equals to " + fMin);
        System.out.println("float fMax equals to " + fMax);
        System.out.println("float fLit equals to " + fLit);
        System.out.println("Size of float is " + Float.SIZE + " bits");
    }
```

Κώδικας 3.5*: Μεταβλητές τύπου float*

Ακολουθεί η έξοδος του κώδικα 3.5

```
float fMin equals to 1.4E-45
float fMax equals to 3.4028235E38
float fLit equals to 3.15
Size of float is 32 bits
```

Προσέξτε πως η σταθερά 3.15 ακολουθείται από το f, εναλλακτικά μπορεί να χρησιμοποιηθεί το F, ώστε να καθοριστεί ο τύπος της σταθεράς ως float. Χωρίς το f ή F, ο τύπος της σταθεράς είναι ο double που παρουσιάζεται αμέσως παρακάτω. Όπως βλέπουμε στην έξοδο του κώδικα 3.5, οι τιμές των fMin και fMax έχουν εκφραστεί με τον επιστημονικό συμβολισμό (scientific notation). Ο επιστημονικός συμβολισμός μπορεί να χρησιμοποιηθεί και για να εκφραστούν κυριολεκτικές σταθερές τύπου float ή double μέσα στον πηγαίο κώδικα. Επομένως, η ελάχιστη πραγματική τιμή που μπορεί να αναπαρασταθεί με τον float είναι 1.4E-45, δηλαδή 1.4x10-45 και η μέγιστη είναι 3.4028235E38, δηλαδή 3.4028235x1038, ο δε χώρος που καταλαμβάνεται στη μνήμη είναι 32 bits.

Οι μεταβλητές τύπου float δύνανται να λάβουν επιπλέον τις ειδικές τιμές Float.NEGATIVE_INFINITY, Float.POSITIVE_INFINITY και Float.NaN (Not a Number). Επιπλέον, σε αντίθεση με τους ακέραιους τύπους, οι πραγματικοί υποστηρίζουν δύο μηδέν, το +0f και το -0f. Αυτό οφείλεται στην κωδικοποίηση IEEE 754 που είναι συμμετρική μεταξύ θετικών και αρνητικών.

Για παράδειγμα, η τιμή Float.NEGATIVE_INFINITY επιστρέφεται όταν ένας αρνητικός αριθμός τύπου float διαιρείται με το +0f ή αντίστοιχα ένας θετικός float διαιρείται με το -0f. Επίσης, Float.NEGATIVE_INFINITY ή Float.POSITIVE_INFINITY μπορεί να επιστραφεί ως αποτέλεσμα έκφρασης στην οποία συμμετέχουν αυτές οι τιμές. Αν όμως έχουμε διαίρεση με Infinity, το αποτέλεσμα είναι NaN. Η τιμή Float.NaN επιστρέφεται και σε διάφορες άλλες περιπτώσεις, όπως για παράδειγμα όταν ζητηθεί η τετραγωνική ρίζα αρνητικού αριθμού.

Η διαίρεση ακεραίου με το 0 παράγει εξαίρεση[5] του τύπου “διαίρεση με το 0” (division by zero). Για παράδειγμα, ο κώδικας που ακολουθεί παράγει εξαίρεση.

```
int i = 1, j = 0;
System.out.println(i / j);
```

Αντίθετα, η διαίρεση πραγματικού με το 0 επιστρέφει Infinity. Για παράδειγμα, ο κώδικας που ακολουθεί τυπώνει Infinity.

```
float f=1, r=0;
System.out.println(f/r);
```

[5] Οι εξαιρέσεις είναι ο μηχανισμός διαχείρισης απροσδόκητων γεγονότων. Παρουσιάζονται αναλυτικά στην ενότητα 16. Για την ώρα θεωρήστε την εξαίρεση συνώνυμη του λάθους χρόνου εκτέλεσης.

Αντίθετα, ο κώδικας

```
float nAN=(float)Math.sqrt(-1);
System.out.println(nAN);
```

θα τυπώσει NaN.

Σημειώστε πως η συνάρτηση sqrt της κλάσης Math επιστρέφει την τετραγωνική ρίζα του ορίσματός της. Στη συγκεκριμένη κλήση επιστρέφει την τετραγωνική ρίζα του -1. Όπως είναι γνωστό, τέτοιος πραγματικός δεν υπάρχει, ως αποτέλεσμα η sqrt επιστρέφει NaN. Επίσης, ο τύπος της τιμής που επιστρέφει η sqrt είναι double. Επομένως, στην κλήση που συζητάμε θα επιστρέψει NaN τύπου double (Double.NaN). Για τον λόγο αυτόν αναγκαζόμαστε να προχωρήσουμε σε μετατροπή του τύπου από double σε float προκειμένου να εκχωρήσουμε την επιστρεφόμενη τιμή στη μεταβλητή nAN τύπου float. Όπως φαίνεται στον κώδικα, για να μετατρέψουμε το αποτέλεσμα της sqrt τοποθετήσαμε πριν την κλήση της το όνομα του νέου τύπου μέσα σε παρενθέσεις. Θα πρέπει να έχουμε υπόψη μας πως μια τέτοια μετατροπή γίνεται με ευθύνη του προγραμματιστή και μπορεί να οδηγήσει σε απώλεια ακρίβειας.

Η κωδικοποίηση IEEE 754 μας δίνει τη δυνατότητα στο ίδιο μήκος μνήμης να αναπαραστήσουμε μεγαλύτερους αριθμούς από ότι η κωδικοποίηση συμπληρώματος ως προς 2 που χρησιμοποιείται για την κωδικοποίηση των ακέραιων. Ωστόσο, είναι σημαντικό να κατανοήσουμε ότι οι πραγματικοί αριθμοί αποθηκεύονται στη μνήμη προσεγγιστικά. Ως αποτέλεσμα συγκρίσεις πραγματικών με χρήση του τελεστή ισότητας (equality operator), ==, ή των υπόλοιπων σχεσιακών τελεστών (>, <, >=, <= και !=, ενότητα 4.1.4) δεν είναι αξιόπιστες και πρέπει να αποφεύγονται.
Για παράδειγμα, ο κώδικας

```
float f=0.1f+0.000000001f;
System.out.println(f==0.1f);
```

θα τυπώσει true.
Προσέξτε τον ακόλουθο κώδικα

```
float d1 = 1, d2=1;
for (int j = 1; j <= 10; j++) {
     d1 = d1+0.00001f;
}
d2=d2+0.00001f * (10);
System.out.println(d1==d2);
```

Οι μεταβλητές d1 και d2 αρχικοποιούνται και οι δύο στην τιμή 1. Σε κάθε βήμα της επαναληπτικής διαδικασίας for, προστίθεται η τιμή 0.00001f στην d1. Έχουμε ένα σύνολο 10 επαναλήψεων, άρα στην d1 προστίθεται 10 φορές το 0.00001. Στη συνέχεια προσθέτουμε στην d2 το 10*0.00001. Αναμένουμε επομένως ότι τα d1 και d2 είναι ίσα μεταξύ τους. Ωστόσο, ο κώδικας θα τυπώσει false, καθώς η τιμή της d1 έχει υπολογιστεί ως 1.0001001 και της d2 ως 1.0001. Για αυτόν τον λόγο, ανάλογα με τις ανάγκες της εφαρμογής μας, η σύγκριση μεταξύ πραγματικών αριθμών μπορεί να γίνεται προσεγγιστικά.

```
static boolean approximateEquals(float f1, float f2, float
epsilon) {
     return Math.abs(f2 - f1) < epsilon;
}
```

Χρησιμοποιήστε τη συνάρτηση approximateEquals για να συγκρίνετε αριθμούς float. Για να δείτε το αποτέλεσμά της, καλέστε την ως εξής:

```
System.out.println(approximateEquals(d1, d2, 0.0001));
```

Η τρίτη παράμετρος συνιστά μια οριακή τιμή. Αν η απόλυτη τιμή της διαφοράς των δύο float είναι μικρότερη από αυτήν την οριακή τιμή, οι αριθμοί θεωρούνται ίσοι. Αν για παράδειγμα θέλουμε να συγκρίνουμε τα μήκη δύο οδών και το μήκος της πρώτης το μετρήσουμε στα 2 km και αυτό της δεύτερης στα 2 km και 1 χιλιοστό του εκατοστού, μπορεί να τις θεωρήσουμε ισομήκεις.

Γενικότερα, ο τύπος float θα πρέπει να αποφεύγεται για διαχείριση τιμών που απαιτούν ακρίβεια. Για τέτοιου τύπου πληροφορίες συνίσταται η χρήση της κλάσης Java.math.BigDecimal. Το ίδιο ισχύει και για τον τύπο double που παρουσιάζεται αμέσως παρακάτω.

double Πρόκειται για τύπο αποθήκευσης πραγματικών αριθμών, επίσης κωδικοποιημένο με το IEEE 754. Η διαφορά του από τον float είναι πως ο double έχει μήκος 64 bits. Πρόκειται για τύπο κινητής υποδιαστολής διπλής ακρίβειας (double-precision 64-bit IEEE 754 floating point). Εξαιτίας του μεγαλύτερου μεγέθους του μπορεί να φιλοξενήσει πραγματικές τιμές με μεγαλύτερη ακρίβεια από τον float. Στην τυπική περίπτωση είναι ο τύπος που θα πρέπει να χρησιμοποιούμε όταν θέλουμε να διαχειριστούμε πραγματικούς αριθμούς. Ο float θα πρέπει να χρησιμοποιείται μόνο στις περιπτώσεις που έχουμε πολλά δεδομένα από πραγματικούς αριθμούς, χρειάζεται να κάνουμε οικονομία στη μνήμη και είμαστε σίγουροι πως η ακρίβεια που προσφέρει, μας αρκεί. Ο τύπος double διαθέτει επίσης τιμές Double.PositiveInfinity, Double.NegativeInfinity, Double.NaN, αρνητικό και θετικό μηδέν με παρόμοια χαρακτηριστικά με τις αντίστοιχες τιμές του float.
Στον κώδικα 3.6 παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου double.

```
    static void doubleDemo() { //Κώδικας 3.6
        double dMin = Double.MIN_VALUE, dMax = Double.MAX_VALUE, dLit =
3.4028235E38;
        System.out.println("double dMin equals to " + dMin);
        System.out.println("double dMax equals to " + dMax);
        System.out.println("double dLit equals to " + dLit);
        System.out.println("Size of double is " + Double.SIZE + "bits");
    }
```

Κώδικας 3.6*: Δήλωση και αρχικοποίηση μεταβλητών τύπου double*

Ακολουθεί η έξοδος του κώδικα 3.6.

```
double dMin equals to 4.9E-324
double dMax equals to 1.7976931348623157E308
double dLit equals to 3.4028235E38
Size of double is 64bits
```

Επομένως, η ελάχιστη τιμή του double είναι 4.9×10^{-324}, η μέγιστη $1.7976931348623157 \times 10^{308}$ και το μέγεθός του 64 bits. Παρατηρήστε πως στην αρχικοποίηση της dLit έχει χρησιμοποιηθεί επιστημονικός συμβολισμός. Οι σταθερές με δεκαδικά που δεν έχουν χαρακτηριστεί ως float θεωρούνται εξ' ορισμού πως είναι τύπου double. Μπορεί να συνοδευτεί μια τέτοια σταθερά από τον χαρακτήρα D ή d προκειμένου να δηλωθεί ο τύπος της σαφώς, ωστόσο αυτό αποφεύγεται από σύμβαση.

Παρότι ο double προσφέρει μεγαλύτερη ακρίβεια από τον float, δεν παύει να αντιπροσωπεύει προσεγγιστικά τους πραγματικούς αριθμούς και η χρήση του θα πρέπει να αποφεύγεται όπου μας ενδιαφέρει η ακρίβεια και στη θέση του να χρησιμοποιείται η κλάση Java.math.BigDecimal. Για προσεγγιστική ισότητα χρησιμοποιήστε τη συνάρτηση

```
static boolean approximateEquals(double d1, double d2,
double epsilon) {
     return Math.abs(d2 - d1) < epsilon;
```

```
    }
```

Ίσως να προσέξατε πως οι συναρτήσεις MIN_VALUE επιστρέφουν αρνητικές τιμές για τους ακέραιους τύπους και θετικές τιμές για τους float και double. Πράγματι, για τους πραγματικούς οι συναρτήσεις αυτές επιστρέφουν την πλησιέστερη τιμή στο 0. Αν χρειάζεστε την ελάχιστη αρνητική τιμή ενός πραγματικού τύπου, χρησιμοποιήστε –Float.MAX_VALUE και –Double.MAX_Value, αντίστοιχα.

boolean Οι μεταβλητές του τύπου boolean λαμβάνουν αυστηρά 2 δυνατές τιμές, true ή false. Ένα δυαδικό ψηφίο αρκεί για την αποθήκευση των τιμών αυτού του τύπου. Ωστόσο, οι προδιαγραφές της Java δεν καθορίζουν το μέγεθος του τύπου. Επομένως, το μέγεθος του τύπου boolean εξαρτάται από τη συγκεκριμένη υλοποίηση. Σε κάποιες γλώσσες είναι δυνατή η μετατροπή μεταξύ boolean και ακέραιων τύπων, στην Java δεν είναι επιτρεπτή τέτοια μετατροπή τύπου [1].
Στον κώδικα 3.7, παρουσιάζουμε παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου boolean:

```
static void booleans() { //Κώδικας 3.7
    boolean b1 = true, b2 = false;
    System.out.println("b1 equals to " + b1);
    System.out.println("b2 equals to " + b2);
}
```

***Κώδικας 3.7**: Δήλωση και αρχικοποίηση boolean μεταβλητών*

Ακολουθεί η έξοδος του κώδικα 3.7.

```
b1 equals to true
b2 equals to false
```

char Ο τύπος char χρησιμοποιείται για την αποθήκευση χαρακτήρων. Υποστηρίζει το σύνολο Unicode, δηλαδή τους χαρακτήρες από τις περισσότερες γλώσσες συν μια ποικιλία χαρακτήρων που δεν συνιστούν γράμματα αλφάβητου και διάφορους χαρακτήρες ελέγχου (control characters) που είναι συνήθως μη εκτυπώσιμοι, όπως για παράδειγμα, ο χαρακτήρας που σηματοδοτεί το τέλος της τρέχουσας γραμμής (End Of Line). Στον κώδικα 3.8 δίνουμε παράδειγμα χρήσης μεταβλητών του τύπου char.

```
1 static void charDemo() { //Κώδικας 3.8
2         char cMin = Character.MIN_VALUE, cMax = Character.MAX_VALUE, cLit =
'A', c = '\u0043';
3         System.out.println(cMin + " " + (int) cMin);
4         System.out.println(cMax + " " + (int) cMax);
5         System.out.println(c + " " + (int) c);
6         System.out.println(cLit + " " + (int) cLit);
7         System.out.println((char) (cLit + 1));
8     }
```

***Κώδικας 3.8**: Παράδειγμα χρήσης του τύπου char*

Ο κώδικας 3.8, έχει την ακόλουθη έξοδο

```
 0
□ 65535
C 67
A 65
B
```